# Python от понятия к решению

Функции и движение данных | Читайте блоки по номерам

## 01 Функция заменяет повторяющийся код

**Концепция.** Функция — именованный блок для одной задачи. def определяет функцию; имя со скобками вызывает её. Определение само по себе не запускает тело.

**Почему так.** Пять копий приветствия пришлось бы править отдельно. Функция оставляет одну реализацию: меняете тело — меняются все вызовы. Это принцип DRY: не повторяйся.

```
def welcome():
    print('Добро пожаловать!')

welcome()
welcome()
```

**Что произойдёт.** Два вызова напечатают приветствие дважды, каждое на своей строке. Просто welcome без () обозначает сам объект функции; тело при этом не выполняется.




**Задача.** Перед каждым отчётом нужно печатать заголовок «Запасы». Объявите show_header() один раз и вызывайте её там, где начинается отчёт.

```
def show_header():
    print('Запасы')

show_header()
```

**Разбор.** def задаёт действие, двоеточие открывает блок, четыре пробела выделяют тело. Последняя строка запускает печать. Имя show_header сообщает читателю, какого действия ожидать.

## 02 Параметры принимают входные данные

**Концепция.** Параметр — имя в определении функции; аргумент — значение при вызове. При позиционной передаче первый аргумент попадает в первый параметр, второй — во второй.

**Почему так.** Функция с зашитым именем «Тоня» годится только для Тони. Параметр отделяет общий алгоритм от данных конкретного пользователя: одна функция обслуживает разные входные значения.

```
def welcome(profession, name):
    print('Привет,', profession, name)

welcome('фермер', 'Тоня')
# Привет, фермер Тоня
```

**На словах.** profession получает строку «фермер», name — «Тоня». Python сопоставляет позиции, а не угадывает смысл слов: если поменять аргументы местами, сообщение тоже изменится.

**Задача.** Рассчитать стоимость 4 банок по 80 рублей. Два входа — количество и цена; переменные снаружи могут называться иначе, чем параметры.

```
def calculate_cost(quantity, unit_price):
    return quantity * unit_price

jars = 4
price = 80
print(calculate_cost(jars, price))
# 320
```

**Разбор.** Значения 4 и 80 передаются в quantity и unit_price; произведение возвращается в print. Не передать обязательный аргумент или передать лишний — TypeError. Сверяйте вызов с определением.

## 03 Умолчания и именованные аргументы

**Концепция.** Значение по умолчанию применяется, когда аргумент пропущен. Именованный аргумент явно указывает получателя: food='инжир'. Это независимые возможности: параметр может быть обязательным и получать значение по имени.

**Почему так.** Частые настройки удобно не повторять, а необычные — переопределять. Имена в вызове помогают не перепутать похожие значения и пропустить необязательный параметр в середине списка.

```
def action(name='Гость', food='ягоды'):
    print(name, 'заготавливает', food)

action()
action(food='инжир', name='Женя')
# Гость заготавливает ягоды
# Женя заготавливает инжир
```

**Задача.** Гость заготавливает яблоки; менять имя не требуется. Вызов action('яблоки') ошибочен по смыслу: единственный позиционный аргумент попадёт в name.

```
action(food='яблоки')
# Гость заготавливает яблоки
```

**Правила для обычных функций курса.** В def сначала обязательные параметры, затем параметры с умолчаниями: def welcome(name, profession='фермер'). В вызове сначала позиционные аргументы, затем именованные: welcome('Тоня', profession='садовод').

**Проверьте себя.** action(name='Женя', 'инжир') даёт SyntaxError; action('Женя', name='Тоня') — TypeError, поскольку name получил два значения. Для именованного аргумента нужно точное имя параметра.

## 04 return отдаёт результат программе

**Концепция.** return выражение вычисляет значение, завершает текущий вызов и отдаёт результат в место вызова. print показывает информацию человеку. Это разные обязанности; их полезно разделять.

**Почему так.** Напечатанное число не становится результатом вызова. Возвращённое число можно сохранить, сложить с другим, передать в функцию и затем вывести.

```
def calculate_cost(quantity, price):
    cost = quantity * price
    return cost

first = calculate_cost(4, 80)
second = calculate_cost(2, 90)
print(first + second)
# 500
```

**Задача и разбор.** Нужно получить общую стоимость двух закупок. Два вызова возвращают 320 и 180; снаружи результаты складываются. Имя cost локально: наружу передаётся значение, а не имя.

```
def show_cost():
    print(320)

result = show_cost()  # Печатает 320.
print(result)         # Печатает None.
```

**Почему появился None.** У show_cost нет return с результатом, поэтому вызов возвращает None. Пустой return тоже возвращает None. Строки после выполненного return в этом вызове недостижимы.

## 05 Вложенные вызовы образуют цепочку

**Концепция.** Результат одной функции может стать аргументом другой. Чтобы вызвать внешнюю функцию, Python сначала вычисляет её аргументы; в примере ниже это означает сначала выполнить внутренний вызов.

**Почему так.** Удобно делить обработку на небольшие понятные шаги: одна функция умножает, другая прибавляет. Каждый шаг можно проверить отдельно. Скобки показывают, как результаты связываются.

```
def multiply_by_three(value):
    return value * 3

def plus_five(value):
    return value + 5

result = plus_five(multiply_by_three(7))
print(result)
# 26
```

**Задача.** Утроить 7 заготовок и добавить ещё 5. Сначала multiply_by_three(7) даёт 21; затем plus_five(21) даёт 26. То же самое в два шага: tripled = multiply_by_three(7), затем result = plus_five(tripled).

**Как выбирать запись.** Короткая цепочка удобна внутри выражения; длинную лучше развернуть, чтобы увидеть промежуточные значения. Для поиска ошибки временно добавьте print(tripled): станет ясно, какой шаг дал неверный результат.

**Проверка понимания.** multiply_by_three(plus_five(7)) вернёт 36: сначала 7 + 5, потом умножение на 3. Менять порядок вложенных вызовов без изменения смысла задачи нельзя.

## 06 От заготовки функции к сообщению

**Концепция.** Функция обязана иметь тело. pass — инструкция «ничего не делать»; ... — значение Ellipsis, которое тоже можно использовать как временную заглушку.

**Почему так.** Заглушка помогает сначала спланировать действия, а реализовать их позже. Она сохраняет структуру, но ещё не решает задачу.

```
def make_message(name):
    pass

print(make_message('Тоня'))
# None
```

**Задача.** Заменить заглушку функцией, которая создаёт приветствие для переданного имени. В PDF курса встречается f-строка: буква f перед кавычкой включает подстановку выражений в фигурных скобках.

```
def make_message(name):
    return f'Добро пожаловать, {name}!'

message = make_message('Тоня')
print(message)
# Добро пожаловать, Тоня!
```

**На словах.** name получает «Тоня»; внутри строки {name} заменяется этим значением; return отдаёт готовый текст. Знак ! напечатается, потому что он явно включён в строку.

**Сравните.** Без f текст '{name}' останется буквальным. f-строка может подставлять и числа: f'Банок: {15}' даёт 'Банок: 15'. Тело с ... без return возвращает None; return ... вернул бы именно Ellipsis. Не путайте заготовку с выполненной реализацией.

## 07 Синтаксис и стиль решают разные задачи

**Концепция.** Синтаксис определяет, может ли Python разобрать код. Стиль помогает человеку читать и менять уже корректную программу. У тренажёра могут быть дополнительные требования к оформлению.

**Почему так.** Пропущенная скобка мешает запуску, а лишний пробел вокруг оператора обычно не меняет вычисление. Различайте причину сбоя и замечание к оформлению.

```
quantity = 4
price = 80
print(quantity * price)
# 320
```

**Задача.** Записать расчёт стоимости и вывести результат. Каждая самостоятельная инструкция стоит на отдельной строке. quantity = 4 print(quantity) на одной строке — SyntaxError.

```
total = (
    quantity * price
    + 2 * 90
)
print(total)
# 500
```

**Разбор переноса.** Открытая скобка показывает, что выражение ещё не закончено; перенос внутри неё допустим. Это одна инструкция на нескольких физических строках. Скобки здесь группируют выражение, а не вызывают функцию.

**Практическое правило.** Не сжимайте код через ; ради экономии места. Сохраняйте логические блоки и пустые строки. Плотная страница шпаргалки не означает, что так же нужно сжимать рабочую программу.

## 08 Отступ задаёт границы блока

**Концепция.** Отступ — часть синтаксиса Python. Строки тела функции находятся глубже строки def. Возврат к прежнему уровню означает выход из блока; произвольный дополнительный отступ может вызвать ошибку.

**Почему так.** Python должен различать тело вызова и внешнюю программу. В курсе используйте 4 пробела на уровень; одинаковый отступ объединяет инструкции блока.

```
def show_report(name):
    print('Отчёт:')
    print(name)

print('Старт')
show_report('Тоня')
# Старт
# Отчёт:
# Тоня
```

**Задача.** Вывести «Старт» один раз, а заголовок и имя — при каждом вызове. print('Старт') стоит снаружи; оба print в теле находятся на одном уровне. Отступы отражают именно такой порядок работы.

**Ошибка по смыслу.** Если передвинуть print('Старт') в тело, сообщение станет повторяться при каждом вызове. Код может работать без исключения, но нарушать условие задачи.

**Ошибка структуры.** Тело без отступа после def приводит к IndentationError. Смешивание табов и пробелов может привести к TabError либо запутать уровни. Настройте Tab в редакторе на вставку четырёх пробелов.

**Проверка.** Комментарии выравнивайте по телу. Именно 4 пробела — соглашение стиля; согласованные уровни — требование языка.

## 09 Пробелы помогают читать выражение

**Концепция.** В обычном присваивании и вокруг арифметических операторов ставят пробелы; после запятой — один пробел. В простых параметрах по умолчанию и именованных аргументах вокруг = пробелы не ставят.

**Почему так.** quantity = 4 читается как отдельное действие, а quantity=4 внутри вызова — как передача одного аргумента. Так легче различать роли знака =; отсутствие пробелов обычно не ломает Python.

```
def calculate_cost(quantity, price=80):
    return quantity * price

result = calculate_cost(4, price=90)
print('Стоимость:', result)
# Стоимость: 360
```

**Задача.** Купить 4 банки по 90 рублей вместо стандартных 80. price=90 относится к конкретному вызову. Внутри result = ... пробелы нужны по обе стороны знака присваивания.

```
# Исполняется, но читать сложнее:
result=calculate_cost(4,price=90)
# Оформление после исправления:
result = calculate_cost(4, price=90)
```

**Ещё два ориентира.** Между именем функции и ( пробела нет: print(result). Сразу после ( и перед ) лишние пробелы не нужны. Пробел после запятой в коде не означает пробел в строковом значении.

**Пустые строки.** В заданиях по модулям курс требует две пустые строки после импортов — соблюдайте условие. Это не правило синтаксиса. В PEP 8 две пустые строки отделяют определения функций верхнего уровня; размещение отступов и пустых строк зависит от контекста.

## 10 Кавычки отделяют текст от кода

**Концепция.** Строку обрамляют прямыми кавычками '...' или "...". Открывающая и закрывающая кавычки должны совпадать. Внешние «ёлочки» не подходят: они могут быть содержимым строки, но не её ограничителями.

**Почему так.** Python использует кавычки, чтобы определить, где заканчивается текст. Неэкранированная внутренняя кавычка того же вида преждевременно закрывает строку, а следующие слова начинают разбираться как код.

```
food = 'инжир'
message = 'Заготовка называется "Инжир".'
print(message)
# Заготовка называется "Инжир".
```

**Задача.** Напечатать название в одинарных кавычках, сохранив внешние одинарные кавычки по стилю курса. Перед внутренними кавычками ставим обратную косую черту: она экранирует символ.

```
message = 'Заготовка называется \'Инжир\'.'
print(message)
# Заготовка называется 'Инжир'.
```

**На словах.** Последовательность обратной косой черты и кавычки обозначает кавычку внутри текста. Сама косая черта в этом выводе не появляется. Можно также выбрать внешние двойные кавычки — Python это допускает.

**Не путайте.** Предпочтение одинарных кавычек — правило приложенного курса, а не запрет Python на двойные. '5' — строка, 5 — число; кавычки влияют и на тип. При склеивании текстов через + пробелы нужно включить в сами строки: 'Женя ' + 'готовит'.

## 11 Комментарий объясняет причину

**Концепция.** Символ # вне строки начинает комментарий до конца строки. Python его не исполняет. Комментарий помогает читателю понять намерение, единицы измерения или неочевидное решение.

**Почему так.** Фраза «умножаем» рядом со знаком * мало помогает. Объяснение «переводим см³ в литры» сохраняет смысл коэффициента и защищает от неправильной замены числа при дальнейшей правке.

```
CM3_TO_LITERS = 0.001

def to_liters(volume_cm3):
    # В одном литре 1000 кубических сантиметров.
    return volume_cm3 * CM3_TO_LITERS

print(to_liters(750))  # Проверяем объём банки.
# 0.75
```

**Задача и разбор.** Для банки 750 см³ требуется получить литры. Коэффициент равен 1 / 1000; комментарий объясняет происхождение числа. Имя volume_cm3 дополнительно фиксирует единицы входа.

**Оформление курса.** После # ставьте пробел. Между кодом и комментарием в конце строки — минимум два пробела. Отдельный комментарий в теле функции выравнивайте по телу. Полные предложения начинайте с большой буквы и завершайте точкой.

**Как читать результат.** print('# Текст') напечатает # Текст: решётка внутри строки — обычный символ. Запись # print('Текст') отключает весь вызов. Комментарии с результатами примеров в шпаргалке даны для проверки и сами не печатаются.

## 12 Хорошее имя предотвращает ошибки

**Концепция.** Используйте осмысленные имена в snake_case: user_name, make_report. Для функций выбирайте действие; для значений — их смысл. В стиле курса: латиница, цифры и _, но не цифра в начале.

**Почему так.** Названия qwerty и asdf заставляют восстанавливать назначение по коду. Имя make_reserve_message сразу говорит, что функция строит текст о заготовке. Имя не должно обещать другой результат.

**Задача.** Исправить пример из урока: Мидас заготавливает золото. Одновременно нужны ясные имена, одинаковые отступы, пробелы в коде и в самом сообщении.

```
def make_reserve_message(name, product):
    return name + ' заготавливает на зиму ' + product

user_name = 'Мидас'
product = 'золото'
message = make_reserve_message(user_name, product)
print(message)
# Мидас заготавливает на зиму золото
```

**Разбор.** Пробелы внутри средней строки не дают словам склеиться. return имеет тот же отступ, что и остальное тело. Имя функции написано строчными буквами с подчёркиваниями.

**Не перекрывайте встроенные имена.** Если написать def print(value): pass, следующие print(...) вызовут вашу пустую функцию. Если написать str = 'текст', вызов str(5) сломается. Переименуйте свои сущности: show_message, text.

**Уточнение.** Python допускает многие Unicode-буквы в именах; латиница — рекомендация курса и командного стиля. Регистр различается: user_name и User_Name — разные имена. Цифра внутри имени допустима: volume_1.

## 13 Модуль даёт готовые инструменты

**Концепция.** Модуль объединяет связанный код: функции, константы и другие объекты. Стандартная библиотека поставляется с Python; сторонние пакеты устанавливают отдельно; пользовательские модули пишет автор проекта.

**Почему так.** Корень числа, календарь и случайные значения уже реализованы. Готовый инструмент экономит работу, а свой модуль позволяет повторно использовать общую логику в нескольких программах. import подключает модуль, но не скачивает его из интернета.

**Разбор.** Например, после import math вызов math.sqrt(256) даёт 16.0; предварительная установка не нужна. print, int, str и type — встроенные функции, они доступны без импорта. «Стандартный модуль» и «встроенная функция» — разные понятия.

**Задача.** Вынести перевод объёма в собственный файл, чтобы использовать его в других программах. Создайте два файла рядом и запустите main.py.

```
# Файл volumes.py.
def to_liters(volume_cm3):
    return volume_cm3 * 0.001

# Файл main.py.
import volumes

print(volumes.to_liters(750))
# 0.75
```

**На словах.** import volumes находит модуль volumes.py, а volumes.to_liters(750) вызывает его функцию. Расширение .py в команде import не пишут. Пример с двумя файлами — практическое развитие идеи пользовательского модуля из урока.

## 14 Точка указывает, где искать имя

**Концепция.** После import math имя math обозначает модуль. Запись math.sqrt обращается к его функции; math.sqrt(9) её вызывает. Запись math.pi получает число: скобки после константы не нужны.

**Почему так.** Точечная нотация делает происхождение инструмента явным. Можно одновременно работать с несколькими модулями и не гадать, откуда взята функция. Такой набор имён называют пространством имён.

```
import math

print(math.sqrt(9))
print(math.pi)
# 3.0
# 3.141592653589793
```

**Задача.** Вычислить объём цилиндрической банки высотой 9 см и радиусом 5 см, пользуясь более точным π из стандартной библиотеки.

```
height = 9
radius = 5
liters = math.pi * height * radius ** 2 * 0.001
print(round(liters, 2))
# 0.71
```

**На словах.** π × h × r² даёт объём в см³; множитель 0.001 переводит его в литры. round(..., 2) округляет результат. Константа pi записана строчными буквами: используйте именно имя из модуля.

**Ловушки.** sqrt(9) после одного import math не определено; math.pi() пытается вызвать число и даёт TypeError. Не присваивайте math = 5 после импорта: это заменит связь имени с модулем.

## 15 from и as меняют доступные имена

**Концепция.** import math as mt связывает модуль с псевдонимом mt. from math import sqrt связывает имя sqrt непосредственно с функцией. После from модульное имя math само по себе в вашем коде не появляется.

**Почему так.** Полное имя math.sqrt ясно показывает источник; прямой импорт сокращает частые вызовы. Псевдоним помогает избежать конфликта или прояснить смысл. Короткое имя полезно только тогда, когда остаётся понятным.

**Задача.** По площади квадратной грядки 81 м² найти сторону. Импортируйте корень под ясным именем и передайте площадь аргументом.

```
from math import sqrt as square_root

area = 81
side = square_root(area)
print('Сторона:', side, 'м')
# Сторона: 9.0 м
```

**Разбор.** При импорте после sqrt нет скобок; при вызове square_root(area) скобки запускают вычисление. Псевдоним не меняет алгоритм. Вариант через модуль: import math as mt, затем mt.sqrt(81) — тоже 9.0.

**Важное уточнение.** from ... import ... выбирает, какие имена появятся в текущем коде; он не гарантирует загрузку только фрагмента модуля или экономию памяти. Выбирайте форму импорта по ясности программы, а не по этому предположению.

## 16 Явные импорты защищают имена

**Концепция.** Из одного модуля можно взять несколько объектов через запятую. Разные модули обычно импортируют на отдельных строках. from module import * добавляет множество имён и скрывает их происхождение.

**Почему так.** У разных инструментов могут совпасть имена. Неявная замена функции меняет поведение кода и осложняет поиск ошибки. Точечная нотация или понятные псевдонимы оставляют связь с источником видимой.

**Что видно.** pow(2, 3) даёт целое 8; после import math вызов math.pow(2, 3) даёт float 8.0. from math import pow перекроет встроенное имя pow в текущей области. Это не ошибка импорта, но изменение значения имени.

**Задача.** Получить корень и число полных банок без неявных имён. Перечислите нужные функции явно; в курсе имена при таком импорте ставят в алфавитном порядке.

```
from math import floor, sqrt

side = sqrt(25)
full_jars = floor(3.8 / 1.2)
print(side, full_jars)
# 5.0 3
```

**Разбор.** Корень 25 равен 5.0; из 3.8 л получаются 3 полные банки по 1.2 л. Другие имена math не становятся доступны автоматически. * обычно импортирует публичные имена, а не буквально каждый внутренний объект модуля.

**Предосторожность.** Не называйте свой файл random.py или datetime.py: он может заслонить одноимённый модуль библиотеки при поиске импорта.

## 17 Выбирайте математическую операцию

**Концепция.** sqrt(x) извлекает корень, math.pow(x, y) возводит в степень с результатом float, floor(x) округляет вниз до целого.

**Почему так.** «Округлить» может означать разные действия. Число полностью заполненных банок нужно округлять вниз; обычное округление до ближайшего целого способно завысить доступное количество.

```
import math

print(math.sqrt(9))     # 3.0
print(math.pow(9, 2))   # 81.0
print(math.floor(3.8))  # 3
print(math.floor(-3.8))  # -4
```

**Задача.** Из 4.7 л сока наполнить банки по 0.5 л. Деление даёт 9.4 банки, но полностью наполнить можно только девять.

```
juice = 4.7
jar_volume = 0.5
full_jars = math.floor(juice / jar_volume)
print(full_jars)
# 9
```

**Разбор.** floor выбирает наибольшее целое, не превосходящее аргумент. Поэтому floor(-3.8) — это -4, а int(-3.8) — -3: int отбрасывает дробную часть к нулю.

**Остатки различаются.** math.remainder(x, y) использует правило IEEE 754 и отличается от %. Пример: math.remainder(3, 2) даёт -1.0, а 3 % 2 — 1. Не выбирайте функцию только по переводу имени.

**Граница применимости.** math.sqrt(-1) даёт ValueError: для вещественного корня нужен неотрицательный аргумент.

## 18 Случайное число и его факториал

**Концепция.** randint(a, b) из random возвращает случайное целое, включая обе границы. factorial(n) из math вычисляет n! — произведение целых чисел от 1 до n; отдельно определено 0! = 1.

**Почему так.** Сохраните случайное число в переменной и вычислите его факториал. Два отдельных вызова генератора могут дать разные числа — тогда сообщение и результат не будут соответствовать друг другу.

**Задача курса.** Выбрать целое от 1 до 10, вычислить факториал и напечатать фразу. При импорте нужны псевдонимы fact и rnd.

```
from math import factorial as fact
from random import randint as rnd

value = rnd(1, 10)
result = fact(value)
print('Факториал', value, 'равен', result)
```




**Один возможный результат:** «Факториал 5 равен 120». Это пример при value = 5, а не гарантированный вывод. При новом запуске значение может отличаться, но должно оставаться в диапазоне 1–10.

**Проверка задачи.** Временно замените value = rnd(1, 10) на value = 5. Ожидается result = 120; остальной код менять не нужно.

**На словах.** 1 × 2 × 3 × 4 × 5 = 120; для восьми различных магазинов 8! = 40320 порядков посещения. factorial требует неотрицательное целое: -1 вызывает ValueError, 5.0 — TypeError. Для итоговой задачи верните случайный выбор.

## 19 Документация объясняет контракт

**Концепция.** Контракт функции — её входы, результат и ограничения. По одному имени нельзя узнать, включён ли конец диапазона, какой тип возвращается или какие значения запрещены.

**Почему так.** Запоминать всю библиотеку не нужно. Надёжнее найти конкретный инструмент в официальном справочнике, прочесть описание и проверить минимальный пример, чем угадывать поведение.

**Задача.** Проверить допустимый диапазон лет datetime. В описании модуля найдите MINYEAR и MAXYEAR; затем импортируйте модуль под псевдонимом dt и прочитайте константы.

```
import datetime as dt

print(dt.MINYEAR, dt.MAXYEAR)
# 1 9999
```

**Разбор.** dt — модуль; MINYEAR и MAXYEAR — числа, поэтому их не вызывают со скобками. Имена нужно переписывать точно, включая регистр. Верхний регистр — стиль этих констант; math.pi имеет другое написание.

**Порядок поиска.** 1. Выберите модуль по задаче. 2. Найдите имя через Ctrl+F или ⌘F. 3. Прочтите сигнатуру, результат, границы и ошибки. 4. Проверьте маленький пример. 5. Встройте его в программу. Не переводите имена Python вместе с английским текстом.

**Дзен Python.** В новом процессе import this печатает набор принципов. Их практический смысл здесь: выбирайте понятные имена, явные импорты и небольшие функции; не игнорируйте сообщения об ошибках. Повторный import уже загруженного модуля обычно не выполняет его тело заново.

**Справочники:** docs.python.org/3/library/math.html; random.html; tutorial/modules.html; PEP 8. Адреса в PDF кликабельны.

## 20 Traceback показывает путь к ошибке

**Концепция.** Traceback — цепочка вызовов, приведших к необработанному исключению. Последняя строка называет тип и причину; выше указаны файл, строка и контекст выполнения.

**Почему так.** Начинайте с последней строки: она сужает поиск. Затем найдите последний относящийся к вашему коду кадр и поднимайтесь к вызывающим функциям. Причина сбоя может находиться раньше указанной строки.

**Задача.** Программа должна печатать название ягоды, но имя при чтении отличается от имени при присваивании:

```
def show_berry():
    red_currant = 'Красная смородина'
    print(redcurrant)

show_berry()
```

```
Traceback (most recent call last):
  File "main.py", line 5, in <module>
    show_berry()
  File "main.py", line 3, in show_berry
    print(redcurrant)
NameError: name 'redcurrant' is not defined
```

**Разбор.** show_berry вызвана из строки 5; в строке 3 Python ищет redcurrant, но в строке 2 было создано red_currant. Это разные имена. Исправление: print(red_currant); результат — «Красная смородина».

**Проверьте причину.** Восстановите нужное имя и снова запустите пример. Не создавайте случайную переменную ради исчезновения ошибки. Выше показан основной текст сообщения для файла main.py; подсказки зависят от версии Python.

## 21 Ошибки синтаксиса и отступов

**Концепция.** SyntaxError означает, что текст не удалось разобрать как программу. IndentationError связан с неверной структурой отступов; TabError — с проблемным смешиванием табов и пробелов.

**Почему так.** Пока не ясны границы строк и блоков, проверять расчёты рано. Сначала исправьте структуру. Указатель может стоять после причины — проверьте и предыдущую строку.

**Типичные причины.** print('Привет) — не закрыта строка; print('Привет' — не закрыта скобка; def hello() — нет двоеточия. Все эти варианты некорректны ещё до выполнения тела программы.

**Задача.** Функция должна вывести начало отчёта, имя и завершение. Приведите все три действия к одному уровню вложенности:

```
def hello(name):
    print('Подготовка к печати...')
    print(name)
    print('Готово!')

hello('Тоня')
# Подготовка к печати...
# Тоня
# Готово!
```

**Разбор.** Если print(name) убрать из тела, отступ перед следующим print может стать неожиданным. Само имя name снаружи недоступно: это параметр вызова.

**Уточнение к уроку.** Строка print('Hello, world!') с обеими кавычками корректна и сама не вызывает ошибку незакрытой строки. Для SyntaxError Python обычно показывает место разбора, а не обычную цепочку исполнявшихся вызовов.

## 22 Тип ошибки подсказывает проверку

**Концепция.** Тип исключения подсказывает проверку. Прочитайте сообщение и проверьте значения, аргументы или импорт, вместо того чтобы менять код наугад.

**NameError:** redcurrant вместо red_currant. Проверьте написание, регистр, видимость и порядок присваиваний.

**TypeError:** 25 + '12' — несовместимые типы; math.pi() — число вызвано как функция; пропущен аргумент. Проверьте type(...) и сигнатуру вызова.

**ValueError:** int('двенадцать') или math.sqrt(-1). Тип аргумента распознан, но значение недопустимо для операции.

**Ошибки импорта:** ModuleNotFoundError — модуль не найден; ImportError при from — возможно, нет импортируемого имени. AttributeError у math.sqtr(9) — нет такого атрибута; нужно sqrt.

**Задача.** Сложить 25 банок ягод и 12 банок овощей, когда число овощей пришло текстом. Сначала проверяем его тип:

```
berries = 25
vegetables = '12'
print(type(vegetables))
# <class 'str'>
total = berries + int(vegetables)
print(total)
# 37
```

**Разбор.** int создаёт число 12, и сложение становится числовым. str(berries) + vegetables дало бы '2512' без исключения, но не сумму. Тип выбирайте по задаче.

**Проверка импорта.** После import math as mt используйте mt.sqrt; после from math import sqrt — sqrt. Сверяйте форму вызова с импортом.

## 23 Отладка проверяет гипотезу

**Концепция.** Баг — ошибка программы; debugging, или отладка, — поиск и устранение причины. Бывают исключения и логические ошибки: программа завершилась, но получила неверный результат.

**Почему так.** Меняя всё сразу, трудно понять, что помогло. Задайте ожидаемый результат, повторите сбой, выдвиньте одну гипотезу, внесите правку и запустите тот же пример.

**Задача.** Для цилиндра h = 9 см и r = 5 см программа считает 0.1413 л, хотя ожидается около 0.71 л. Ошибки Python нет — неверна формула.

```
height = 9
radius = 5
wrong = 3.14 * height * radius * 0.001
print(wrong)
# 0.1413
correct = 3.14 * height * radius ** 2 * 0.001
print(round(correct, 2))
# 0.71
```

**Разбор.** Сравнение с $V = \pi \times h \times r^2$ обнаруживает потерянный квадрат радиуса. Гипотеза: вместо $r^2$ использовано r. Замена одной части формулы подтверждает её. При r = 1 ошибка скрылась бы: 1 и $1^2$ совпадают.

**Контроль после правки.** Проверьте ещё один радиус, единицы измерения и типы входных данных. Удалите временные диагностические print перед отправкой, если задание требует точного вывода. Для случайных данных сначала используйте фиксированный вход.

**Самопроверка.** '25' + '12' даёт '2512'; int('25') + int('12') даёт 37. Обе записи исполняются: правильность определяет требование задачи. Ошибка — рабочая информация, а не оценка ваших способностей.

## 24 Когда тренажёр не принимает решение

**Концепция.** Различайте ошибку решения и сбой платформы. Запуск без исключения ещё не доказывает соблюдение условия и формата вывода.

**Задача-сценарий.** Факториал вычисляется верно, но ответ отклонён. Сверьте с условием имена, импорты, псевдонимы, границы диапазона, пробелы вывода и лишние print.

**Почему сначала сохранить код.** Перезагрузка может вернуть исходный шаблон. До неё скопируйте полное решение в текстовый файл. Оно пригодится и поддержке.

**Порядок из приложенного FAQ.** 1. Сохраните код. 2. Выполните принудительное обновление: Windows — Ctrl+F5, macOS — ⌘Shift+R. 3. Вставьте сохранённый код и повторите проверку. Такое обновление не равно очистке всего хранилища браузера.

**Проверка расширений.** В FAQ рекомендован обновлённый Chromium-браузер. Проверьте чистый профиль или инкогнито без расширений: некоторые разрешены и там. Если сбой исчез, включайте их по одному, чтобы найти виновника; затем настройте исключение для сайта или отключите его.

**Если нужна поддержка.** Подготовьте ссылку на задачу, полный код, скриншоты ошибки и консоли. В Chromium: Ctrl+Shift+J в Windows, ⌘Option+J в macOS. Используйте чат поддержки платформы.

```
Задача: [ссылка]
Ожидал: [результат]; получил: [ошибка]
Проверил: [шаги и их результаты]
Прилагаю: полный код и скриншоты.
```

**Основа шпаргалки:** шесть вложений о функциях, модулях, правописании, ошибках и работе платформы. Шаги для тренажёра изложены по приложенному FAQ.

СВЯЗЬ И НОВЫЕ МАТЕРИАЛЫ

# Где искать новые материалы

Новые файлы, а также ссылки на сообщества во ВКонтакте, Telegram, Teletype и Boosty — и, возможно, на YouTube, VK Видео, Rutube, Stepik и Udemy — ищите по этим контактам.

| | |
|---|---|
| TELEGRAM | t.me/artur_khant |
| ВКОНТАКТЕ | vk.ru/artur_khant |
| TELETYPE | teletype.in/@artur_khant |
| INSTAGRAM | instagram.com/artur_khant |
| GITHUB | github.com/artur-khant |
| САЙТ-РЕЗЮМЕ | arturishkildin.cv |

Не нашли ссылку на хранилище файлов или нужный цифровой продукт? Напишите мне в личные сообщения или просто наберите в поиске **АРТУР ХАНТ**.